

Laura Dib Lelis
Mariana Sun
Vitor Custódio
Rafael Cordova
Arthur Carvalho

A01:2021 - BROKEN ACCESS CONTROL

*conceito, impactos, detecção e mitigação de falhas de
controle de acesso*

RESUMO

Este artigo trata da categoria A01:2021 do OWASP Top 10, Broken Access Control, apontada pela própria OWASP como a falha mais recorrente entre as aplicações testadas para o levantamento de 2021. O texto descreve o conceito da vulnerabilidade, os impactos que ela causa em aplicações e negócios, as formas usuais de detectá-la e as práticas recomendadas para preveni-la. Como parte da pesquisa, o grupo construiu uma aplicação de laboratório que simula um portal acadêmico, com uma falha real de controle de acesso. Um aluno autenticado consegue chegar ao painel do professor e alterar a nota de outro aluno, porque o sistema confere apenas se existe uma sessão ativa, sem checar se o papel dessa sessão é de professor. A aplicação tem alternância ao vivo entre um modo vulnerável e um modo corrigido. Primeiro a falha é explorada, depois a mesma tentativa é bloqueada quando o servidor passa a validar o papel da sessão antes de responder.

Palavras-chave: segurança da informação; controle de acesso; OWASP Top 10; autorização; controle de acesso por função; IDOR; elevação de privilégio; segurança de aplicações web; sessão de usuário; Flask.

INTRODUÇÃO

Broken Access Control é a categoria do OWASP Top 10 que reúne falhas ligadas ao controle de acesso, casos em que um usuário consegue acessar ou alterar um recurso que não deveria, seja dado de outro usuário, seja função reservada a outro papel. Um usuário comum consegue acessar dados de outro usuário, alterar um pedido que não é dele, ou chegar a uma função de administrador só porque o sistema nunca checou se ele tinha permissão para aquilo.

Ela virou a vulnerabilidade número 1 do Top 10 em 2021, subindo da quinta posição que ocupava em 2017. A mudança vem de uma base de mais de 500 mil aplicações testadas ao todo: dessas, 94% apresentaram algum tipo de falha de controle de acesso, e a categoria concentra o maior número de CWEs¹ (Common Weakness Enumeration) mapeados entre todas as do Top 10, um total de 34.

¹ CWE é a sigla de Common Weakness Enumeration, um catálogo mantido pela MITRE que lista e numera tipos genéricos de falha de software. Cada CWE descreve um padrão de vulnerabilidade, não um caso específico: CWE-22 é Path Traversal, CWE-284 é Improper Access Control, CWE-639 é IDOR, e assim por diante.

Principais formas em que essa falha aparece:

- Escalonamento horizontal (IDOR): um usuário acessa dados de outro usuário do mesmo nível de permissão.
- Escalonamento vertical de privilégios: um usuário comum chega a funções de administrador.
- Bypass de controle de acesso: acesso direto a uma página ou função restrita sem passar pela checagem de permissão.
- Path traversal (directory traversal): manipulação de caminho de arquivo para ler algo fora do diretório permitido.

Não é uma falha rara ou de nicho. É a mais recorrente entre aplicações reais.

CONCEITO DA VULNERABILIDADE

Broken Access Control é a falha que aparece quando um sistema confirma corretamente quem é o usuário, mas nunca confirma o que aquele usuário específico pode acessar. Na definição da OWASP (OWASP Top 10:2021, categoria A01), controle de acesso é o mecanismo que garante que alguém só consiga fazer aquilo que sua permissão autoriza; quando esse mecanismo falha, o resultado costuma ser divulgação de informação não autorizada, alteração ou destruição de dados, ou a execução de uma função fora do que era permitido àquele usuário (OWASP FOUNDATION, 2021).

A chave para entender o conceito é a diferença entre autenticação e autorização. Autenticação responde "quem é você", geralmente via login, senha ou token de sessão. Autorização responde uma pergunta diferente, "o que você pode fazer", e só é checada depois que a identidade já foi provada. É exatamente nessa segunda etapa que mora a falha de Broken Access Control: o sistema autentica corretamente, mas nunca confirma se aquele usuário autenticado tem permissão sobre o recurso específico que está pedindo, seja um número de conta na URL, um arquivo, ou uma função administrativa (OWASP FOUNDATION, 2021).

Essa gravidade aparece também em publicações acadêmicas recentes. Almushiti et al. (2024) apontam Broken Access Control como a vulnerabilidade mais explorada atualmente pelos atacantes entre as categorias do OWASP Top 10: uma exploração bem-sucedida pode dar ao atacante privilégios administrativos e comprometer toda a aplicação web. Jan et al. (2026) chegam à mesma conclusão ao analisar casos reais de

ataque: o controle de acesso deficiente continua sendo uma das causas mais recorrentes de comprometimento em sistemas web.

IMPACTOS NO NEGÓCIO/APLICAÇÃO

Os impactos variam de acordo com o que o atacante consegue alcançar depois de explorar a falha, então não existe um único cenário de dano. Numa aplicação financeira, a ausência de checagem de posse num endpoint de extrato ou transferência pode expor saldo e movimentação de qualquer cliente, ou até permitir uma operação em nome de terceiros. Num sistema acadêmico ou de saúde, a mesma falha expõe notas, prontuários ou dados pessoais de qualquer usuário cadastrado. O dado sensível não exige nenhuma técnica sofisticada para ser alcançado: só em 2021, a OWASP registrou 318.487 ocorrências desse tipo de falha, o maior volume entre todas as categorias do Top 10 (OWASP FOUNDATION, 2021).

Dois casos reais mostram esse impacto fora do ambiente de teste. A First American Financial Corp. expôs 885 milhões de documentos de transações imobiliárias em 2019 porque bastava trocar um número sequencial na URL para acessar o registro de outro cliente, sem autenticação nenhuma (KREBS, 2019). O Correio americano (USPS) expôs dados de 60 milhões de usuários em 2018 pelo mesmo padrão, numa API do serviço Informed Delivery (KREBS, 2018). No Brasil, esse tipo de vazamento carrega um risco a mais: o dado exposto costuma ser pessoal, o que transforma o incidente também numa questão de conformidade com a LGPD (BRASIL, 2018), somando custo jurídico e reputacional ao custo técnico da correção.

FORMAS DE DETECÇÃO

Diferente de uma falha como SQL Injection, que deixa uma assinatura reconhecível para scanner automático, Broken Access Control depende da regra de negócio de cada aplicação. Uma ferramenta genérica não sabe, sozinha, se a matrícula 2023002 pertence ao usuário que está logado, ela só percebe o problema se comparar respostas entre sessões diferentes, e a maioria dos scanners não faz isso por padrão. É por isso que a detecção dessa categoria depende muito mais de teste manual e revisão de código do que de varredura automatizada (OWASP FOUNDATION, 2021).

O OWASP Web Security Testing Guide (WSTG) organiza esse teste manual em cinco frentes: travessia de diretório e inclusão de arquivos, verificação de bypass no esquema de autorização, teste de elevação de privilégio, teste de referências diretas a objetos inseguras (IDOR) e teste de falhas em fluxos OAuth (OWASP FOUNDATION, 2020). Na prática, isso significa criar duas ou mais contas de teste com papéis diferentes, autenticar com uma delas e tentar acessar, alterar ou excluir recursos que pertencem à outra, tanto trocando identificadores em URLs quanto reenviando requisições capturadas com um proxy interceptador. Na revisão de código, o ponto de atenção é toda rota que recebe um identificador de recurso via URL, corpo ou query string: cada uma delas precisa de uma checagem explícita de posse antes de responder (OWASP FOUNDATION, 2021). Registro de tentativas negadas também ajuda depois que o sistema já está em produção, tanto para pegar bug quanto para ataque em andamento (OWASP FOUNDATION, 2021).

BOAS PRÁTICAS DE MITIGAÇÃO

A recomendação central da OWASP para essa categoria é negar por padrão. Qualquer recurso que não seja explicitamente público deve exigir uma decisão de acesso positiva antes de ser entregue, nunca o contrário. Essa checagem precisa acontecer no servidor, nunca só no cliente, porque qualquer verificação feita em JavaScript no navegador, ou escondida na interface, pode ser contornada por quem manda a requisição direto para a API (OWASP FOUNDATION, 2021).

Um ponto que costuma passar despercebido é a diferença entre checar o tipo do objeto e checar o objeto específico. Não basta o sistema confirmar que o usuário tem permissão para "ver notas" de forma genérica, ele precisa confirmar que aquele usuário específico tem permissão sobre aquele recurso específico, comparando o identificador pedido com o identificador, ou o papel, vinculado à sessão autenticada. Na aplicação descrita neste trabalho, essa é exatamente a linha que separa o modo vulnerável do modo corrigido, a rota do painel do professor passou a validar, a cada requisição, se o papel da sessão ativa é "professor", em vez de aceitar qualquer sessão autenticada.

A OWASP recomenda ainda outras medidas. Centralizar a lógica de controle de acesso num único ponto da aplicação, em vez de reimplementar rota por rota, evita que cada rota nova vire uma chance de esquecer a checagem. Identificadores internos previsíveis também devem ser evitados quando possível, com valores aleatórios ou mapeamentos indiretos no lugar deles quando a exposição do identificador real não puder ser evitada. Sessões precisam ser invalidadas no logout, com tokens de vida curta, e o sistema deve limitar a taxa de requisições para dificultar tentativas automatizadas sobre identificadores sequenciais. Por fim, quando uma tentativa de acesso é negada, o sistema deve falhar de forma segura, sem devolver mensagem de erro que revele detalhe interno (OWASP FOUNDATION, 2021).

Para sistemas com regras de permissão mais complexas que um simples "aluno ou professor", vale considerar modelos baseados em atributos ou em relacionamento (ABAC² ou ReBAC³) em vez de depender só de papéis fixos (RBAC). Modelos baseados apenas em papel tendem a crescer de forma descontrolada conforme o sistema ganha exceções e casos particulares, um problema conhecido como role explosion.

DEMONSTRAÇÃO PRÁTICA DE EXECUÇÃO E EXPLORAÇÃO

Para a demonstração prática deste projeto, em vez de apresentar a correção só como um trecho de código isolado, optamos por construir um site funcional que reproduz, em ambiente controlado, o mesmo tipo de falha discutida ao longo deste artigo. A ideia foi permitir que a exploração e a correção da vulnerabilidade fossem observadas na prática, em tempo real, e não apenas descritas em teoria.

O site simula um portal acadêmico fictício, com duas contas de aluno e uma conta de professor. Cada aluno deveria enxergar só o próprio boletim, e apenas o professor deveria ter acesso ao painel que reúne as notas de todos os alunos e permite alterá-las. A aplicação foi desenvolvida em Python com o framework Flask, com todo o controle de sessão e autenticação implementado do zero, para fins didáticos, sem bibliotecas prontas de autorização.

A falha de Broken Access Control foi introduzida deliberadamente na rota responsável pelo painel do professor. Essa rota confere apenas se existe uma sessão autenticada, sem checar qual é o papel (aluno ou professor) associado a ela. Na prática, um aluno autenticado que acessa diretamente o endereço do painel do professor consegue não só ver o boletim de qualquer colega, como também alterar o valor das notas, mesmo nunca tendo feito login como professor.

Para tornar a correção visível ao vivo, o site inclui um alternador entre os dois modos, vulnerável e corrigido, sem precisar reiniciar o servidor. No modo corrigido, a mesma rota passa por uma checagem adicional, além de exigir sessão autenticada, o sistema confere se o papel dessa sessão é de fato "professor". Ao repetir a mesma tentativa de acesso depois dessa correção, o aluno recebe uma resposta de acesso

2 ABAC é controle de acesso baseado em atributos. Em vez de dar permissão por papel fixo ("professor pode isso, aluno pode aquilo"), o sistema olha as características do momento: quem é o usuário, qual recurso ele quer, em que contexto. Um exemplo: "só libera se o usuário for professor E for o professor daquela turma E estiver dentro do horário de aula". No RBAC, essa regra viraria um papel novo só para esse caso. No ABAC, ela é só mais uma condição.

3 ReBAC (Relationship-Based Access Control) é o modelo de controle de acesso em que a permissão depende do vínculo específico entre o usuário e o recurso, como "autor de" ou "matriculado em", em vez de depender apenas do papel do usuário ou de atributos genéricos da sessão

negado (código HTTP 403⁴), tanto para visualizar quanto para alterar qualquer nota que não seja a sua própria.

Como reproduzir a demonstração

O passo a passo abaixo permite executar a aplicação localmente e repetir os mesmos testes descritos neste trabalho:

1. Ter os arquivos do projeto disponíveis em uma pasta local.
2. Criar um ambiente virtual Python e instalar a dependência Flask⁵.
3. Executar o servidor local com o comando `python app.py`.
4. Acessar o endereço <http://127.0.0.1:5050/site-demonstracao-ld/> no navegador.
5. Autenticar-se com a conta de um aluno e tentar acessar diretamente a rota do painel do professor, alterando o identificador do colega na URL. Nesse ponto aparece a falha, o boletim de outro aluno aparece na tela e pode ser alterado.
6. Autenticar-se como professor e acionar o alternador de modo, ativando a correção.
7. Repetir a tentativa de acesso indevido como aluno. A aplicação passa a responder com o código HTTP 403, bloqueando tanto a leitura quanto a alteração dos dados de outro usuário.

Os testes foram feitos em ambiente local, com dados fictícios, e cada etapa foi documentada, testada e executada.

4 HTTP 403 é o código que um servidor manda de volta quando entende o seu pedido, mas se recusa a atender porque você não tem permissão para isso. É diferente do 404, que é "não existe", ou do 401, que é "você nem provou quem é ainda". No seu site, é exatamente o que acontece: o aluno está autenticado (o servidor sabe quem ele é), só que não tem o papel de professor, então a rota nega o acesso com 403 em vez de mostrar a nota de outro aluno.

5 Flask é tipo o "encanamento" do site. Ele pega o que chega pela URL, manda para a função certa no seu código, e devolve a resposta para o navegador. Sem ele, você teria que escrever esse trâmite todo na mão.

CONCLUSÃO

A pesquisa confirma o que os próprios números da OWASP já sugeriam, Broken Access Control não é uma falha exótica, nem exige ferramentas sofisticadas para ser explorada. Na demonstração construída, um aluno autenticado chegou ao painel do professor só trocando o identificador do colega na URL, e a partir daí conseguiu tanto ver quanto alterar a nota de outro aluno, porque a rota só checava se existia uma sessão ativa, sem confirmar se o papel daquela sessão era de professor.

É sempre a mesma causa. O sistema resolve corretamente a pergunta "quem é você" no login, mas nunca chega a fazer a segunda pergunta, "o que você pode fazer", na hora de entregar o recurso. É essa lacuna, não a complexidade do ataque, que explica por que Broken Access Control lidera o levantamento da OWASP. Qualquer rota que aceite um identificador vindo do cliente, ou que presuma o papel de quem está pedindo, é candidata a essa falha, a menos que alguém lembre de checar a posse do recurso e a permissão do papel antes de responder.

Isso ficou claro também na correção. Não foi preciso nenhum produto de segurança adicional para resolver o problema. Bastou uma mudança pequena, localizada numa única função, que passou a validar o papel da sessão antes de liberar o acesso ao painel do professor. Com a proteção ligada, a mesma tentativa que antes devolvia o boletim de outro aluno passou a ser bloqueada com erro 403, sem quebrar nenhuma funcionalidade legítima do sistema. A literatura da OWASP recomenda justamente isso como a defesa mais consistente contra essa categoria de falha: checagem de autorização no servidor, aplicada rota por rota.

REFERÊNCIAS

ALMUSHITI, Elaf; ZAKI, Raseel; THAMER, Nora; ALSHAYA, Rima. An Investigation of Broken Access Control Types, Vulnerabilities, Protection, and Security. In: *Advances in Emerging Information and Communication Technology (ICIEICT 2023)*. Cham: Springer, 2024. p. 253-269. Disponível em: https://doi.org/10.1007/978-3-031-53237-5_16. Acesso em: 22 set. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 18 set. 2026.

JAN, Sadeeq; KHAN, Safi Ullah; WAHAB, Abdul; MOHAMMAD. Access Restricted: A Study of Broken Access Control Vulnerabilities. *Archives of Advanced Engineering Science*, v. 4, n. 3, p. 316-321, 2026.

Disponível em: <https://doi.org/10.47852/bonviewAAES52024016>. Acesso em: 22 set. 2026.

KREBS, Brian. First American Financial Corp. Leaked Hundreds of Millions of Title Insurance Records. Krebs on Security, 24 maio 2019. Disponível em: <https://krebsonsecurity.com/2019/05/first-american-financial-corp-leaked-hundreds-of-millions-of-title-insurance-records/>. Acesso em: 18 set. 2026.

KREBS, Brian. USPS Site Exposed Data on 60 Million Users. Krebs on Security, 21 nov. 2018. Disponível em: <https://krebsonsecurity.com/2018/11/usps-site-exposed-data-on-60-million-users/>. Acesso em: 18 set. 2026.

OWASP FOUNDATION. A01:2021 – Broken Access Control. In: OWASP Top 10:2021. Disponível em: https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/index.html. Acesso em: 12 set. 2026.

OWASP FOUNDATION. Authorization Cheat Sheet. In: OWASP Cheat Sheet Series. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html. Acesso em: 12 set. 2026.

OWASP FOUNDATION. Insecure Direct Object Reference Prevention Cheat Sheet. In: OWASP Cheat Sheet Series. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html. Acesso em: 21 set. 2026.

OWASP FOUNDATION. WSTG - v4.2. Web Security Testing Guide, 2020. Disponível em: <https://owasp.github.io/www-project-web-security-testing-guide/v42/>. Acesso em: 21 set. 2026.